INSURANCE AI AGENT • TECHNICAL SPECIFICATION

Comprehensive RAG Architecture, Pre-Retrieval Filtration & Semantic Vector Pipeline

A Deep-Dive Reference into Ingestion Guardrails, AI Gatekeeper Stratification, Tabular Disaggregation, ONNX Embedding, and Hybrid Qdrant Re-Ranking

System: Autonomous Insurance Policy Knowledge & FNOL Copilot | **Architecture:** Hybrid Qdrant RAG | **Document Date:** October 2026 | **Version:** 2.0 Production Ready

1. Executive Overview & Core RAG Philosophy

Retrieval-Augmented Generation (RAG) within the regulated insurance sector requires significantly stricter deterministic precision than generic enterprise document question-answering. In insurance underwriting and claims adjudication, hallucinations regarding compulsory excess amounts, medical exclusions, or FNOL (First Notice of Loss) submission deadlines result in regulatory non-compliance, financial loss, or unlawful claim rejections.

To solve these challenges, this system implements a **Zero-Hallucination, Multi-Tiered RAG Pipeline**. Rather than passing raw, unverified documents directly to an embedding model, the architecture applies a series of strict pre-retrieval validation filters, domain-aware chunking windows, tabular catalogue disaggregation, local ONNX dense vector embeddings, and an 8-signal hybrid re-ranking algorithm.

> ***Core Principle:*** *No document reaches the vector store without cryptographic deduplication, multi-point stratified authenticity verification, and domain density scoring. Every retrieved clause must maintain exact traceability to its policy code, section, and exclusion boundary.*

High-Level Pipeline Stages

Pipeline Stage	Core Technologies	Key Functional Output
1. Pre-RAG Ingestion Guardrails	SHA-256 Hashing, Mammoth, pdf-parse, SheetJS	Rejection of non-insurance documents, duplicate blocking, text sanitization.
2. AI Gatekeeper Verification	Stratified Sampling, Groq Llama-3, Domain Density Analyzer	Authenticity score (0.0-1.0), policy type classification, test-data override.
3. Chunking & Disaggregation	Regex Section Splitter, Tabular Offer Parser	Contextual pre-header injection, disaggregates spreadsheets into independent offers.
4. Local Dense Vectorization	ONNX Runtime, all-MiniLM-L6-v2 (q8 quantized)	384-dimensional dense vectors generated locally on CPU with zero token cost.
5. Vector Storage & Payloads	Qdrant Vector Database, Cosine Distance	RFC4122 v4 deterministic UUIDs, metadata payload indexing in policy_documents.
6. Query Contextualization	Conversational Topic Resolver, Pronoun Expansion	Resolves multi-turn follow-ups ('is theft excluded?' -> merges prior policy codes).
7. Hybrid Retrieval & Re-ranking	Qdrant Semantic Search + 8 Multi-Signal Boosts	Exact clause boosts (+15.0), exclusion boosts (+6.0), threshold filtering.
8. Grounded Generation & Audit	Groq Llama-3.3-70B, Strict Grounding Prompt	Legally grounded response with verbatim clause citations and cost calculation.

2. Pre-RAG Filtration: Ingestion Guardrails & AI Gatekeeper

A critical flaw in standard RAG implementations is allowing irrelevant, duplicate, or corrupted documents into the vector database. If an employee uploads a vendor invoice, a cooking recipe, or a software tutorial, standard vector search will corrupt insurance retrieval. The system solves this with **five sequential filtration layers** before any vector indexing occurs.

Layer 1: Cryptographic Bit-Exact SHA-256 Deduplication

Every incoming file stream is immediately passed through Node.js crypto SHA-256 hashing before writing to permanent storage. If a file hash matches an existing indexed record in MongoDB, the upload is blocked immediately. This avoids redundant vector clustering and stops duplicate clauses from competing for cosine similarity slots.

Layer 2: Multi-Format Extraction & Row Serialization

Supports PDF (via pdf-parse), Microsoft Word (mammoth for clean semantic HTML-to-text extraction), plain text, and Excel spreadsheets (.xlsx, .xls, .csv via SheetJS). For spreadsheets, the parser flattens 2D tabular grids into readable, header-paired semantic clause blocks: SECTION: SheetName - Row X | Header: Value.

Layer 3: 4-Point Stratified Document Sampling

Insurance policies often span 50 to 200 pages. Sending the entire document to an LLM gatekeeper is slow and causes token overflow. Truncating to just the first 1,000 words is equally flawed because exclusions and FNOL rules appear at the end. The system introduces **Stratified 4-Point Sampling**:

- **Offset 0.00 (Preamble / Schedule):** Captures policy title, underwriter identity, policyholder details, and initial definitions.
- **Offset 0.33 (Coverage Sections):** Captures standard insured perils (accidental damage, third-party liability, inpatient care).

• **Offset 0.66 (Exclusions & Deductibles):** Captures critical exclusions (pre-existing conditions, wear-and-tear, intoxication).

• **Offset 0.90 (Claims & FNOL Procedures):** Captures notification deadlines, document requirements, and dispute clauses.

Layer 4: Insurance Domain Density Analyzer

To protect against false rejections (such as benchmark policies labeled with *'FICTIONAL TEST DATA FOR SOFTWARE TESTING'*), the service runs a deterministic keyword density evaluator. It computes the frequency of 12 distinct insurance terms:

> ***Density Formula:*** *Scan text for key indicators: [annual cover limit, pre-existing condition, waiting period, excess / deductible, inpatient, outpatient, claims process, underwriting, premium, endorsement, fnol, sum insured].*
> ***Override Rule:*** *If matchCount ≥ 4 AND LLM rejected solely due to fictional disclaimer → **OVERRIDE REJECTION**, assign Authenticity = 0.95, and accept document!*

Layer 5: LLM Authenticity & Category Classification

The composite stratified sample is evaluated by Groq Llama-3 under a strict JSON enforcement prompt. The LLM must output: { isValidInsuranceDocument: boolean, confidenceScore: float, policyType: enum, reasoning: string }. Documents with confidence score < 0.65 or invalid domain categories are rejected at the gate, keeping the vector store pristine.

3. Domain-Aware Semantic Chunking & Tabular Disaggregation

Standard recursive character text splitters break sentences indiscriminately, often splitting an exclusion list in half or separating a deductible from its applicable section. The system uses two specialized parsing engines:

A. Insurance Contract Semantic Clause Parser

Modeled after legal regulatory standards, the chunker uses regex boundary detection targeting structural insurance markers: SECTION [0-9]+, PART [A-Z], SCHEDULE, EXCLUSIONS, CONDITIONS, and ENDORSEMENTS.

• **Sliding Window Overlap:** Chunks are sized to 500 tokens with a 75-token overlap to ensure clause context is never truncated.

• **Contextual Pre-Header Injection:** Every chunk has its hierarchy prepended directly into its vector text:
[DOCUMENT: Royal Sun Car Policy] [POLICY CODE: POL-AUTO-2026] [TYPE: AUTO] [SECTION: Section 1 - Loss or Damage].
This guarantees the embedding vector retains full document lineage even when isolated.

B. Tabular Spreadsheet Disaggregation Engine

When multi-product spreadsheets (e.g., 30_fictional_insurance_offers.xlsx) are ingested, flattening them into one giant text blob creates four major operational failures: (1) single category dropdown forces Health and Motor together; (2) individual policies have no database identity; (3) searching for Policy #1 retrieves random chunks from Policy #2; and (4) an underwriter cannot update Policy #1 without re-uploading the entire file.

To solve this, policyOfferService executes **Two-Tier Disaggregation**:

1. **Master Record:** Creates parent PolicyDocument with isBatchCatalogue = true and totalOffersCount = 30.

2. **Granular Offer Records:** Iterates through all sheet rows. For each row, maps columns (Plan Name, Insurer, Sum Insured, Deductible, Waiting Period, Exclusions) into an individual PolicyOffer MongoDB document.

3. **Standardized Policy Codes:** If the spreadsheet contains an ID (e.g. INS001), it is standardized. If missing, the backend generates an enterprise code: POL-[CATEGORY]-[ROW_NUM]-[DOC_SEED] (e.g. POL-HLTH-001).
4. **Dedicated Vector Points:** Every single offer gets its own vector embedding in Qdrant with full metadata payloads.

4. Local Vector Embedding Engine & Qdrant Topology

Unlike systems relying on remote embedding APIs (such as OpenAI text-embedding-3-small), this project runs a **100% Local, In-Process ONNX Embedding Engine** via @huggingface/transformers.

Specification	Technical Value / Configuration
Model Architecture	Xenova/all-MiniLM-L6-v2 (Sentence Transformers)
Quantization Format	8-Bit Quantized (q8 ONNX model execution)
Vector Dimensionality	384-dimensional dense float vector
Pooling & Normalization	Mean pooling with L2 unit normalization (||v|| = 1.0)
Batch Streaming Size	32 chunks per batch with event-loop setImmediate yielding
API Dependency & Cost	Zero remote API dependency, $0.00 embedding token cost
Data Privacy Standard	Strict zero-egress: policy wording never leaves the host server
Vector DB Collection	policy_documents (Cosine Distance Metric)

Deterministic RFC4122 v4 UUID Generation

Qdrant requires vector point IDs to be valid UUIDs or positive integers. Rather than assigning random UUIDs (which would make re-indexing and deduplication difficult), point IDs are derived deterministically: MD5(parentDocumentId + '_' + policyReferenceCode + '_' + chunkIndex) converted into an RFC4122 v4 UUID. This enables deterministic point upserts and targeted point updates.

5. Query Contextualization & Multi-Signal Hybrid Re-Ranking

Standard semantic search fails when users ask conversational follow-up questions (e.g. *'what is the excess for that?'* or *'is theft excluded?'*). The query lacks the noun 'motor insurance' or the policy code. The system executes a two-stage retrieval process:

Stage 1: Conversational Topic & Policy Code Injection

The function buildContextualSearchQuery analyzes the last 4 conversation turns. If it detects follow-up triggers (pronouns 'this', 'that', 'it', or short questions under 55 characters), it extracts previously mentioned policy codes (POL-[0-9]+, SECTION [0-9]+) or previous user topics and merges them into the query string before vectorization.

Stage 2: 8-Signal Hybrid Re-Ranking Algorithm

Once Qdrant returns candidate vectors (top 20), they are re-ranked using domain-specific mathematical scoring adjustments:

Re-Ranking Signal	Score Delta	Evaluation Criteria & Target
Exact Policy / Clause Code	**+15.0**	Query mentions exact code (e.g. 'INS001', 'SECTION 1', 'POL-1168'). Guarantees requested clause is ranked #1.

Numerical Code Match	**+10.0**	Substring code match (e.g. user types '001' and clause code contains 'INS001').
Exclusion Clause Prioritization	**+6.0**	Triggered when query contains 'not covered', 'exclusion', 'excluded', 'exception', or 'denied'.
Excess / Deductible Prioritization	**+5.5**	Triggered when query asks about 'excess', 'deductible', 'out of pocket', or 'compulsory cost'.
FNOL & Claims Process Prioritization	**+4.5**	Triggered when query asks about 'file claim', 'fnol', 'timeline', 'hours', 'accident report'.
Policy Category Alignment	**+2.5**	Matches line of business (e.g. 'Health', 'Motor', 'Property', 'Accident').
Domain Discriminator Keywords	**+2.0 each**	Specific terms: *subrogation, indemnity, write-off, collision, windscreen, pre-existing, waiting period*.
Table of Contents Demotion	**-2.5**	Penalizes non-informative outline chunks, index headers, or chunks with fewer than 80 characters.

6. Semantic Caching & Auditable Answer Generation

To optimize latency and minimize LLM API token consumption, the system integrates a **Qdrant Semantic Prompt Cache** and an auditable generation framework with Groq Llama-3.

High-Speed Semantic Prompt Caching (< 15ms Latency)

A separate collection, semantic_cache, indexes incoming user questions. When a new query arrives, it is embedded and searched against the cache at a cosine similarity threshold of ≥ 0.88. If a match occurs, the cached verified answer is returned immediately. This eliminates redundant Groq LLM API calls and delivers instant sub-second responses.

Legally Grounded LLM Response Generation

When generating answers, the top-ranked clauses are formatted into a strict citation context. The system instruction enforces:

1. **Zero Extrapolation:** Ground responses strictly in retrieved excerpts. State 'Not specified in policy' if clauses are silent.
2. **Exclusion Highlighting:** Always explicitly separate coverage from policy exclusions.
3. **Exact Monetary Figures:** Report compulsory and voluntary excess amounts verbatim.
4. **Mandatory Citations:** Every answer cites the source policy name and clause code: [SOURCE: INS001 - Harbor Essential Health].

Real-Time Token Usage & Financial Cost Auditing

Every RAG query calculates token consumption and financial cost using calculateTokenCost:

```
Cost Formula:
Prompt Cost = (Prompt Tokens / 1,000,000) * $0.59
Completion Cost = (Completion Tokens / 1,000,000) * $0.79
Total Cost = Prompt Cost + Completion Cost
Metrics Reported: latencyMs, promptTokens, completionTokens, totalCost
```

7. Granular Single-Policy Editing & Live Vector Re-Sync

In production insurance systems, policy excess amounts, limits, and endorsements change frequently. Under traditional RAG architectures, modifying one policy required re-uploading and re-embedding the entire 30-offer spreadsheet. This system introduces **Targeted Vector Synchronization**:

- **REST Endpoint:** PUT /api/policy-offers/:id allows updating any policy offer (e.g. changing excess from £250 to £200).
- **Database Mutation:** Updates the specific PolicyOffer document in MongoDB.
- **Selective Re-Embedding:** Reformats only that offer's semantic text and generates a fresh 384-dimensional vector via ONNX.
- **Point Upsert:** Upserts the point to Qdrant at its deterministic UUID point ID (offer.qdrantPointId).
- **Zero Side-Effects:** All other 29 policy offers and vectors remain untouched and online without downtime.

Summary Conclusion: *The Insurance AI Agent implements an enterprise-ready, defensible RAG pipeline. From pre-ingestion SHA-256 deduplication and 4-point stratified AI Gatekeeper validation, to tabular disaggregation, local zero-cost ONNX vector embeddings, and an 8-signal hybrid re-ranking algorithm, the system guarantees accurate, auditable, and sub-second insurance intelligence.*